

Reviewer Pack — Minimal Number of Ternary Diatomic Sequences and SAT Clause ...

Entry 8cb37d685d27 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 21:36:10 UTC

Minimal Number of Ternary Diatomic Sequences and SAT Clause Subset Complexity

Entry ID: 8cb37d685d27 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 21:36:10 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Number Theory (Ternary Diatomic Sequences) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subset Complexity)

Statement:

For every Boolean satisfiability instance with m clauses over n variables, the number of ternary diatomic sequences required to represent all possible truth assignments is upper-bounded by a polynomial in n and m , i.e., $|D(\text{SAT_instance})| \leq O(n^c * m^d)$ for some constants c and d .

Rationale (proposer's reasoning):

Ternary diatomic sequences are a combinatorial object that can be used to represent Boolean functions. Their number might provide insights into the structure of SAT instances and their complexity. A connection between this mathematical object and SAT clause subset complexity could lead to new algorithms for solving SAT or understanding its inherent difficulty.

Taxonomy category: ternary_diatomic_sequences (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 05cfab429d654a9a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given Boolean satisfiability instance, if the number of distinct ternary diatomic sequences $|D(\text{SAT_instance})|$ exceeds $O(n^c * m^d)$, where n is the number of variables and m is the number of clauses, then the conjecture is falsified. Otherwise, if $|D(\text{SAT_instance})|$ does not exceed this bound for all tested instances, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('ternary diatomic sequences' AND 'Boolean satisfiability') - ('SAT clause subset complexity' AND 'combinatorial number theory') - ('polynomial upper bound' AND 'representation of truth assignments' AND ('ternary diatomic sequences' OR 'Boolean satisfiability'))

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            while len(set(clause)) == 1: # Ensure at least one positive and one negative
                clause[random.randint(0, n-1)] *= -1
            clauses.append(clause)
        return clauses

    def ternary_diatomic_sequences(clauses):
        sequences = set()
        for assignment in product([-1, 0, 1], repeat=len(clauses)):
            sequence = []
            for i, clause in enumerate(clauses):
                if all(assignment[j] * clause[j] >= 0 for j in range(len(clause))):
                    sequence.append(1)
                elif any(assignment[j] * clause[j] < 0 for j in range(len(clause))):
                    sequence.append(-1)
                else:
                    sequence.append(0)
            sequences.add(tuple(sequence))
        return sequences
```

```

def product(iterables):
    pools = [tuple(pool) for pool in iterables]
    result = [[]]
    for pool in pools:
        result = [x + [y] for x in result for y in pool]
    return result

n_max = 0
total_metric_value = 0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in range(5, 41):
    for m in range(5, 41):
        if n * m > 200: # Avoid excessive computation time
            continue
        instances_tested += 1
        n_max = max(n_max, n)
        clauses = generate_sat_instance(n, m)
        sequences = ternary_diatomic_sequences(clauses)
        metric_value = len(sequences)
        total_metric_value += metric_value

        if instances_tested >= 30:
            break

    if instances_tested >= 30:
        break

mean_metric_value = total_metric_value / instances_tested
support_fraction = instances_tested / 60

return {
    "metric_name": "Number of Ternary Diatomic Sequences",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='not supported' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d1798c3.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d1798c3.py", line 64, in run_trial
    sequences = ternary_diatomic_sequences(clauses)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d1798c3.py", line 32, in ternary_diatomic_sequences
    for assignment in product([-1, 0, 1], repeat=len(clauses)):
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: run_trial.<locals>.product() got an unexpected keyword argument 'repeat'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Review and fix the error in the test code to ensure it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12982
2	propose	ollama_remote	glm4:latest	0	0	12983
3	preregistration	ollama_remote	glm4:latest	0	0	9930
4	novelty	ollama_remote	glm4:latest	0	0	14015
5	novelty	ollama_remote	glm4:latest	0	0	8846
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25782
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8867

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23653
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12471
10	judge	ollama_remote	glm4:latest	0	0	30810

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160340 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/8cb37d685d27.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/8cb37d685d27.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/8cb37d685d27.tar.gz* (if generated)